



# PRIM'S ALGORITHM

---

PREPARED BY  
**SWAPNIL BISWAS**



# GREEDY ALGORITHM

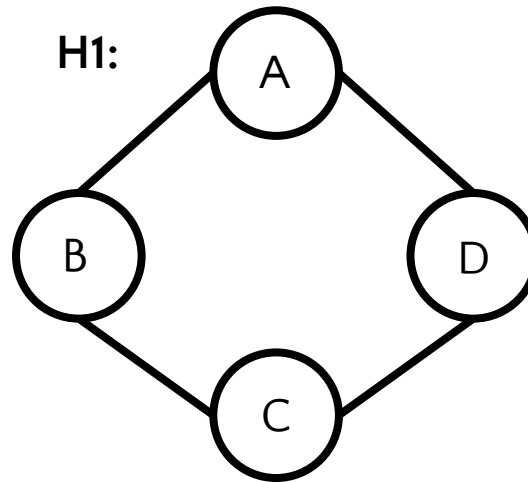
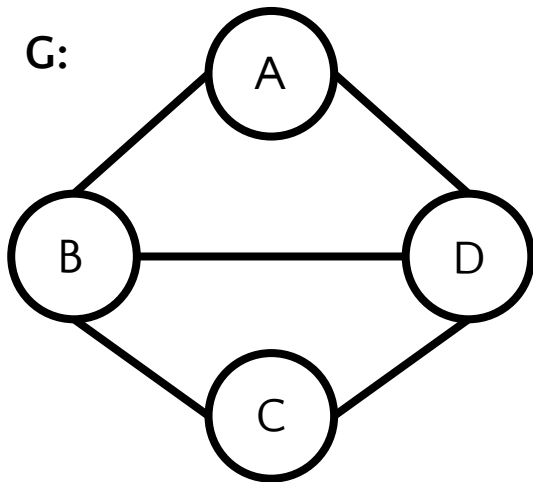
---

- ❑ Always takes the best at each phase
  - Without regards for future consequences
- ❑ Hoping that by choosing a local optimum at each step, it will end up at a global optimum
- ❑ If there are multiple options at any phase that indicates multiple global optimum
- ❑ Can be applied if a problem has optimal substructure property
- ❑ Can not be applied if a problem has overlapping sub problem property
- ❑ Calculating Minimum Spanning Tree of a graph is solved by Greedy approach

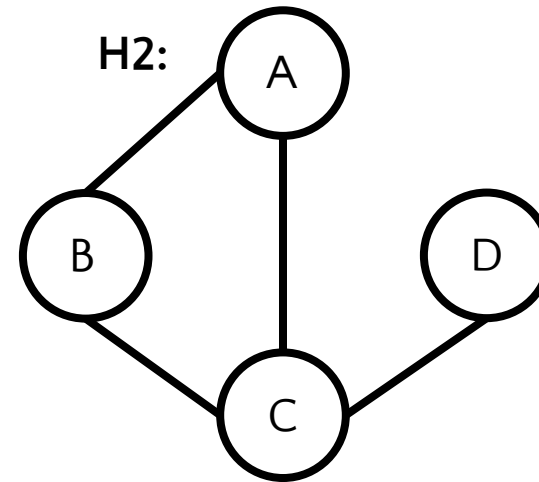
# SPANNING SUB GRAPH

□ H is a spanning sub graph of G if

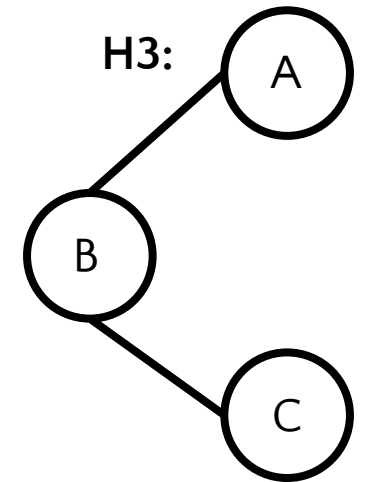
- $V(H) = V(G)$
- $E(H) \subseteq E(G)$



YES



NO



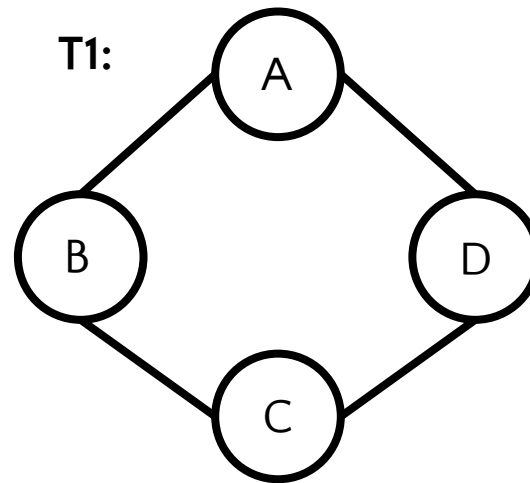
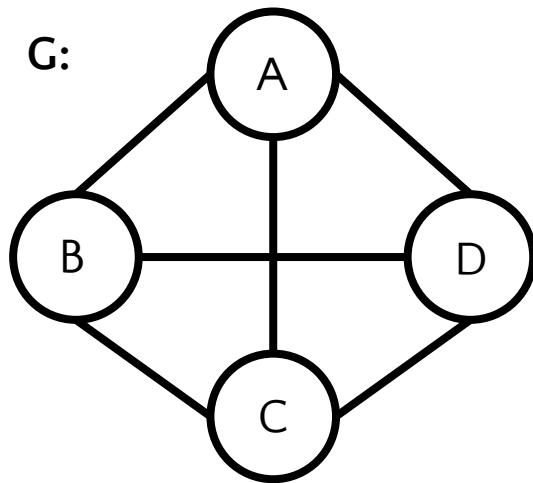
NO

H1 is a spanning sub graph of G but H2 and H3 is not

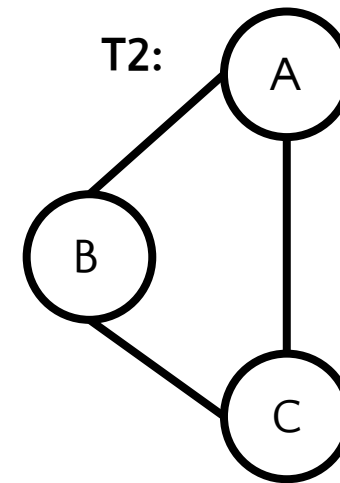
# SPANNING TREE

□ T is a spanning sub tree of G if

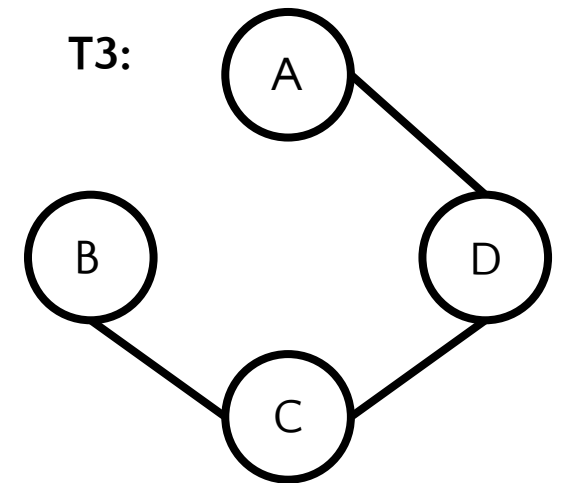
- $V(T) = V(G)$
- $E(T) \subseteq E(G)$
- T is a tree



NO



NO



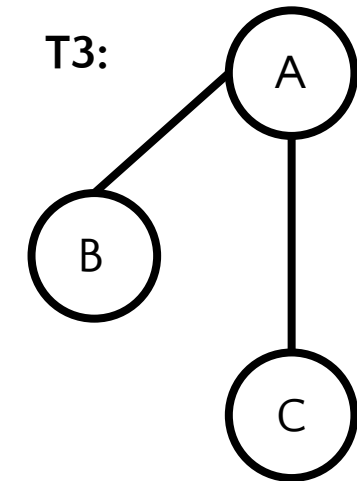
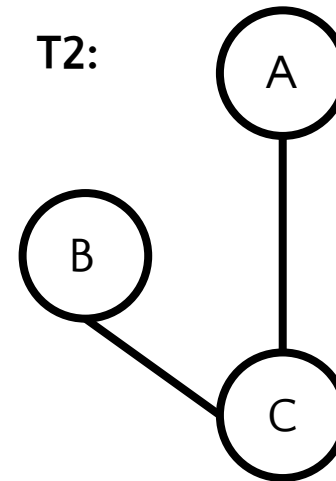
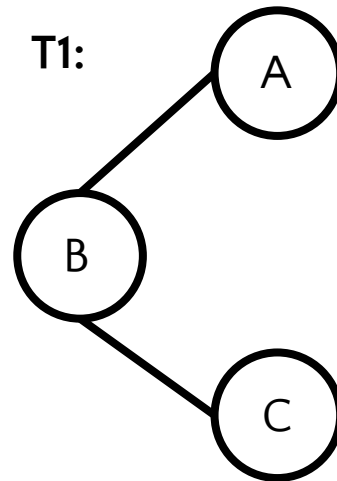
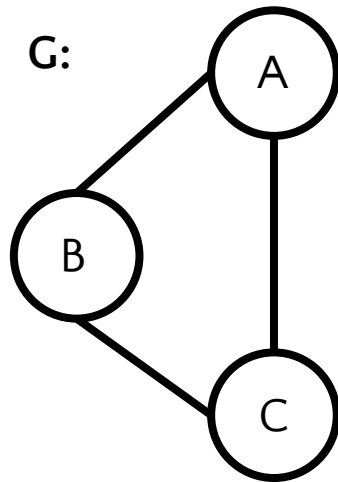
YES

T3 is the only Spanning Tree of G



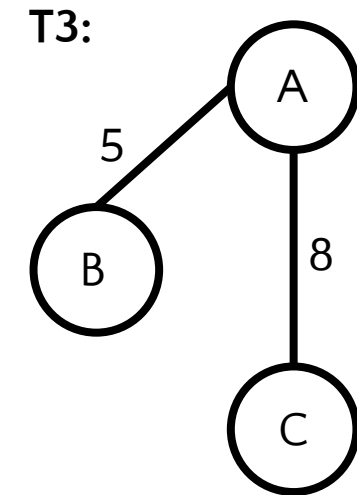
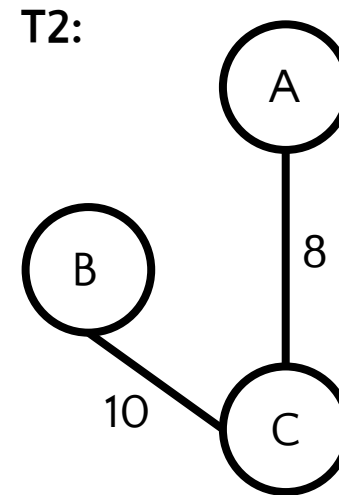
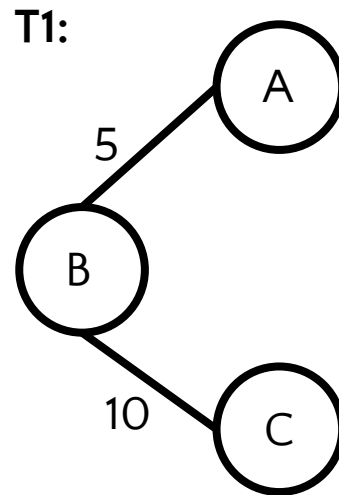
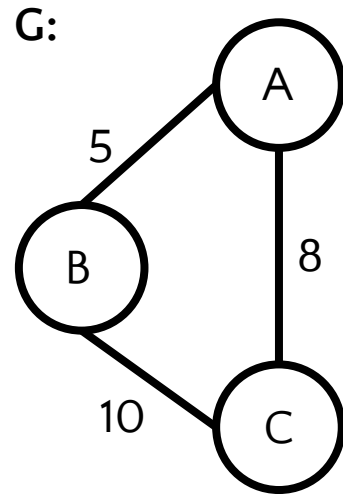
# MULTIPLE SPANNING TREE

- A graph can have multiple spanning trees



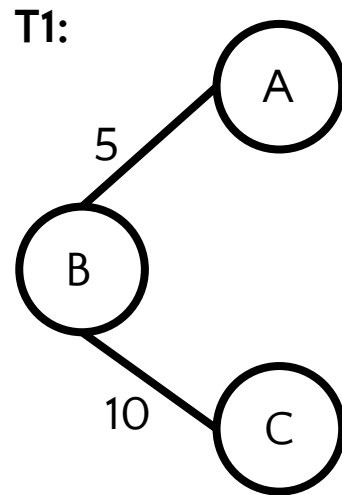
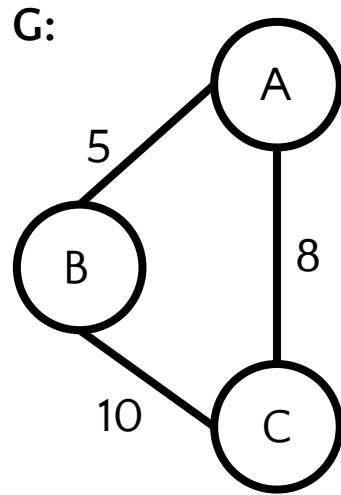
# WEIGHTED SPANNING TREE

- Edges are weighted

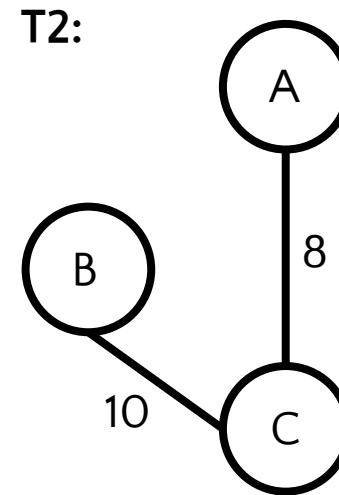


# MINIMUM SPANNING TREE (MST)

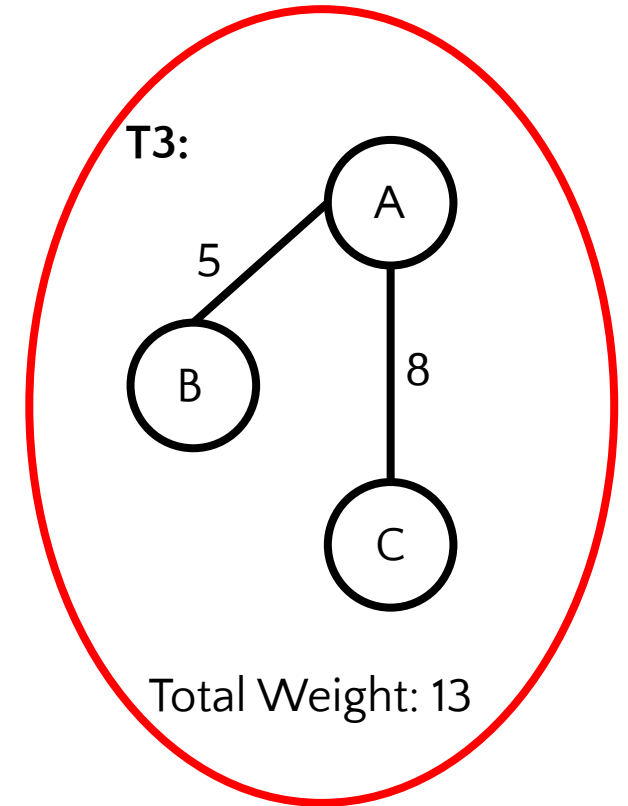
- The spanning tree of G having minimum weight sum



Total Weight: 15



Total Weight: 18

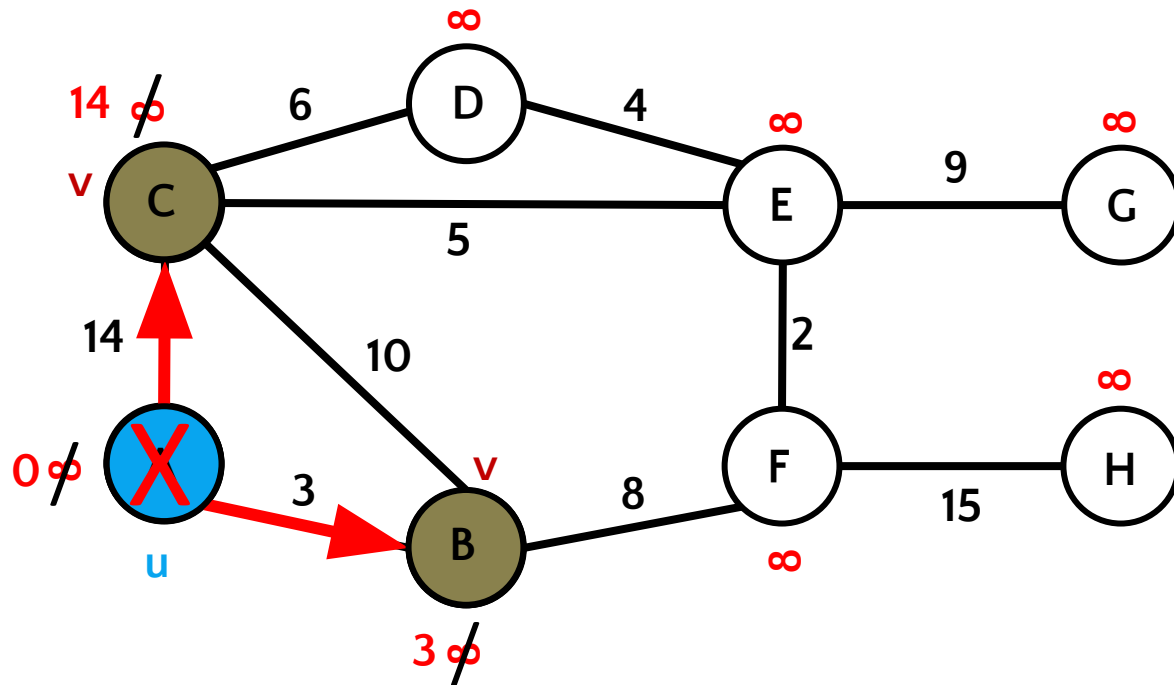


Total Weight: 13

**T3 is the Minimum Spanning Tree of G**



# PRIM's ALGORITHM (SIMULATION)

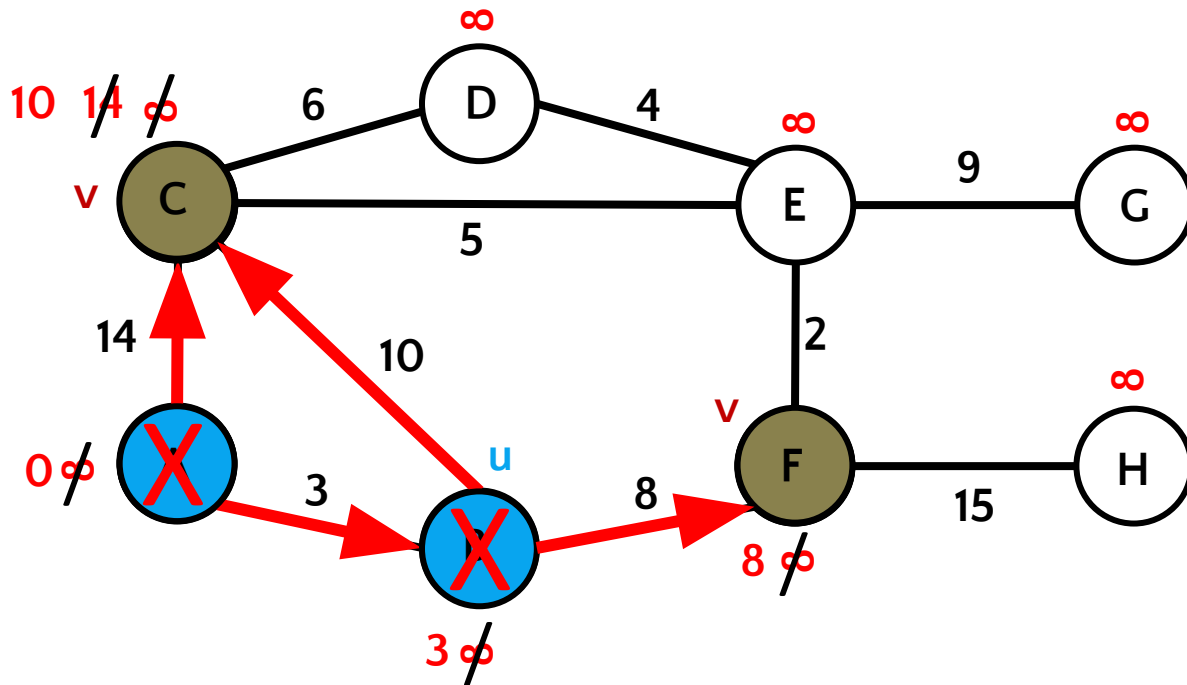


- Set the key of all vertices to INF
- Pick a random vertex as root and do  $\text{key}[\text{root}] := 0$
- Iteration until all vertices are visited
  - $u :=$  unexplored vertex with lowest key
  - $v :=$  unexplored adjacent vertices of  $u$
  - If  $\text{key}(v) > w(u,v)$ 
    - $\text{parent}(v) := u$
    - $\text{key}(v) := w(u,v)$
  - Make  $u$  explored



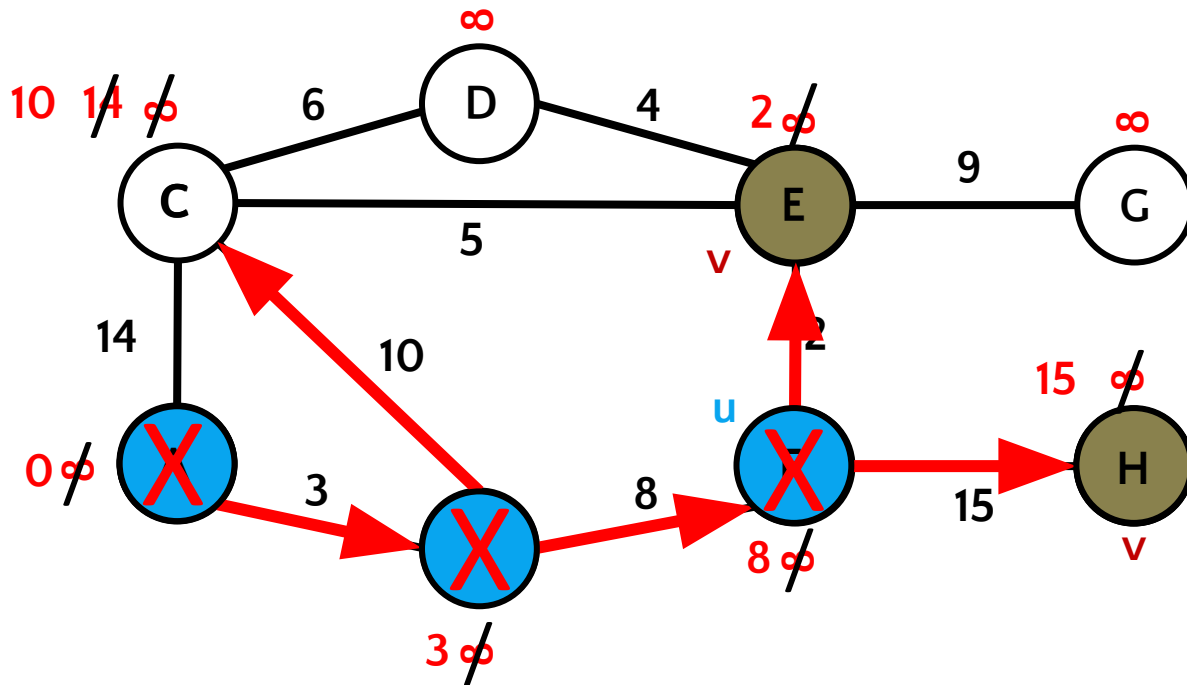


# PRIM's ALGORITHM (SIMULATION)



- Set the key of all vertices to INF
- Pick a random vertex as root and do  $\text{key}[\text{root}] := 0$
- Iteration until all vertices are visited
  - $u :=$  unexplored vertex with lowest key
  - $v :=$  unexplored adjacent vertices of  $u$
  - If  $\text{key}(v) > w(u,v)$ 
    - $\text{parent}(v) := u$
    - $\text{key}(v) := w(u,v)$
  - Make  $u$  explored

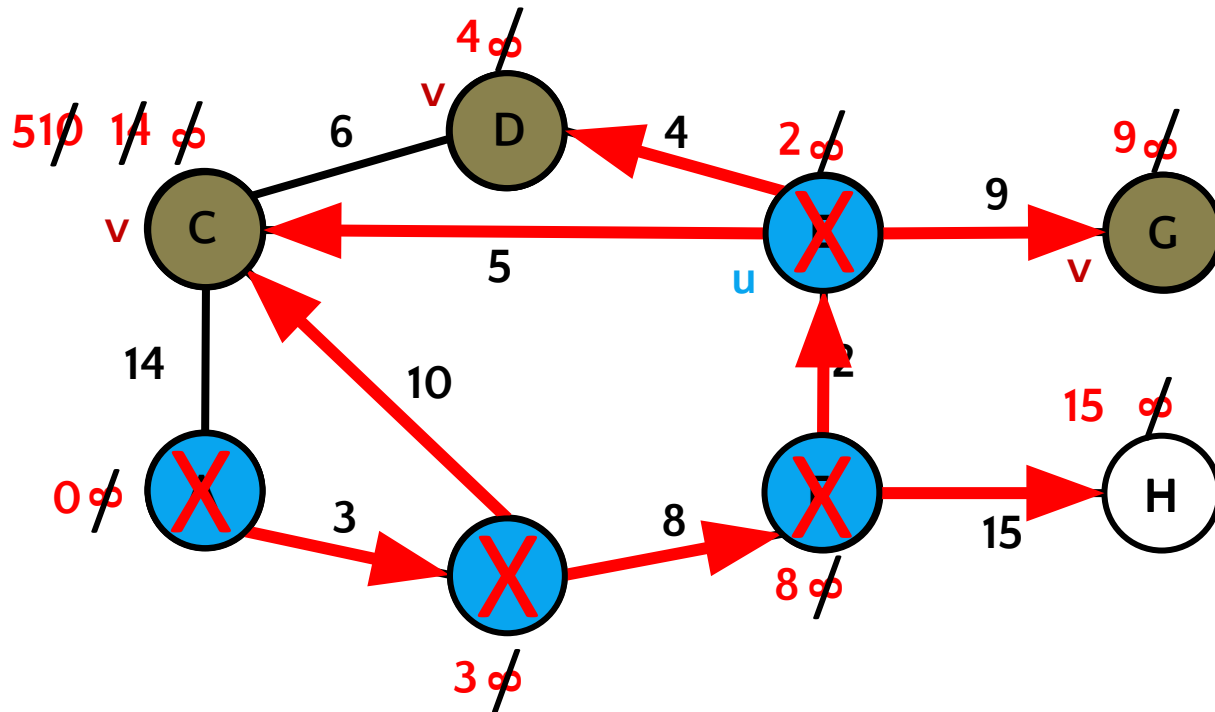
# PRIM's ALGORITHM (SIMULATION)



- Set the key of all vertices to INF
- Pick a random vertex as root and do  $\text{key}[\text{root}] := 0$
- Iteration until all vertices are visited
  - $u :=$  unexplored vertex with lowest key
  - $v :=$  unexplored adjacent vertices of  $u$
  - If  $\text{key}(v) > w(u, v)$ 
    - $\text{parent}(v) := u$
    - $\text{key}(v) := w(u, v)$
  - Make  $u$  explored



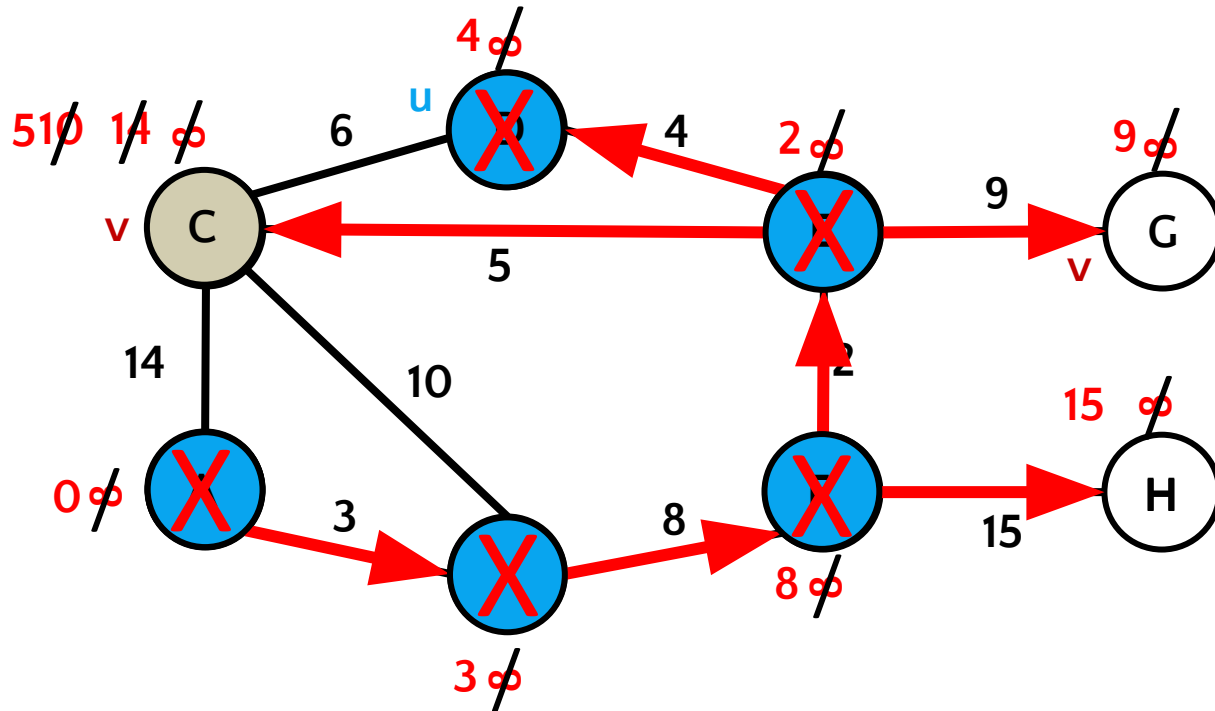
# PRIM's ALGORITHM (SIMULATION)



- Set the key of all vertices to INF
- Pick a random vertex as root and do  $\text{key}[\text{root}] := 0$
- Iteration until all vertices are visited
  - $u :=$  unexplored vertex with lowest key
  - $v :=$  unexplored adjacent vertices of  $u$
  - If  $\text{key}(v) > w(u,v)$ 
    - $\text{parent}(v) := u$
    - $\text{key}(v) := w(u,v)$
  - Make  $u$  explored

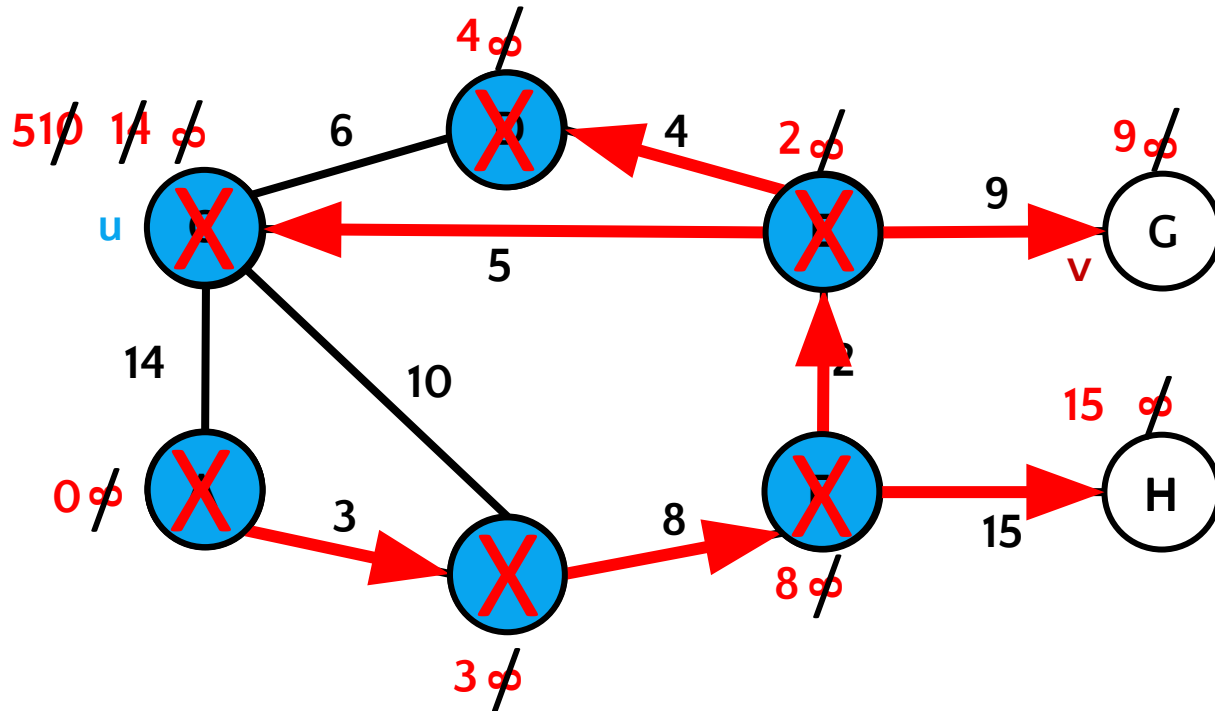


# PRIM's ALGORITHM (SIMULATION)



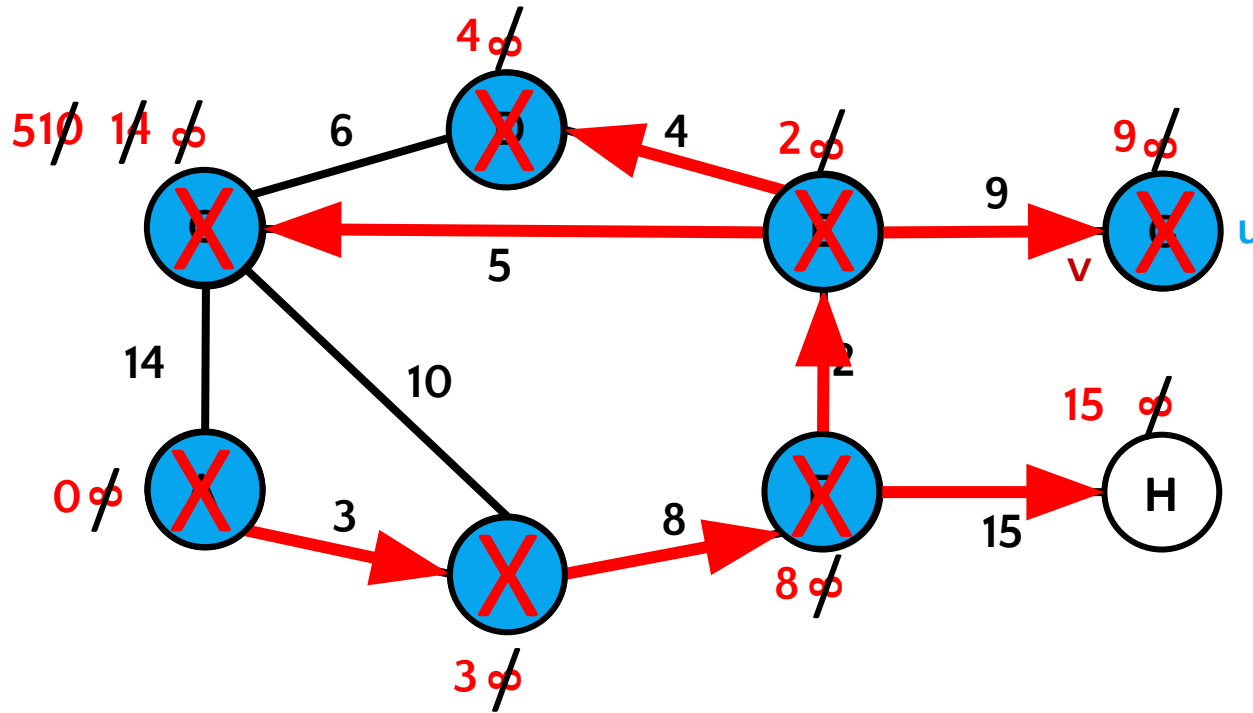
- Set the key of all vertices to INF
- Pick a random vertex as root and do  $\text{key}[\text{root}] := 0$
- Iteration until all vertices are visited
  - $u :=$  unexplored vertex with lowest key
  - $v :=$  unexplored adjacent vertices of  $u$
  - If  $\text{key}(v) > w(u, v)$ 
    - $\text{parent}(v) := u$
    - $\text{key}(v) := w(u, v)$
  - Make  $u$  explored

# PRIM's ALGORITHM (SIMULATION)



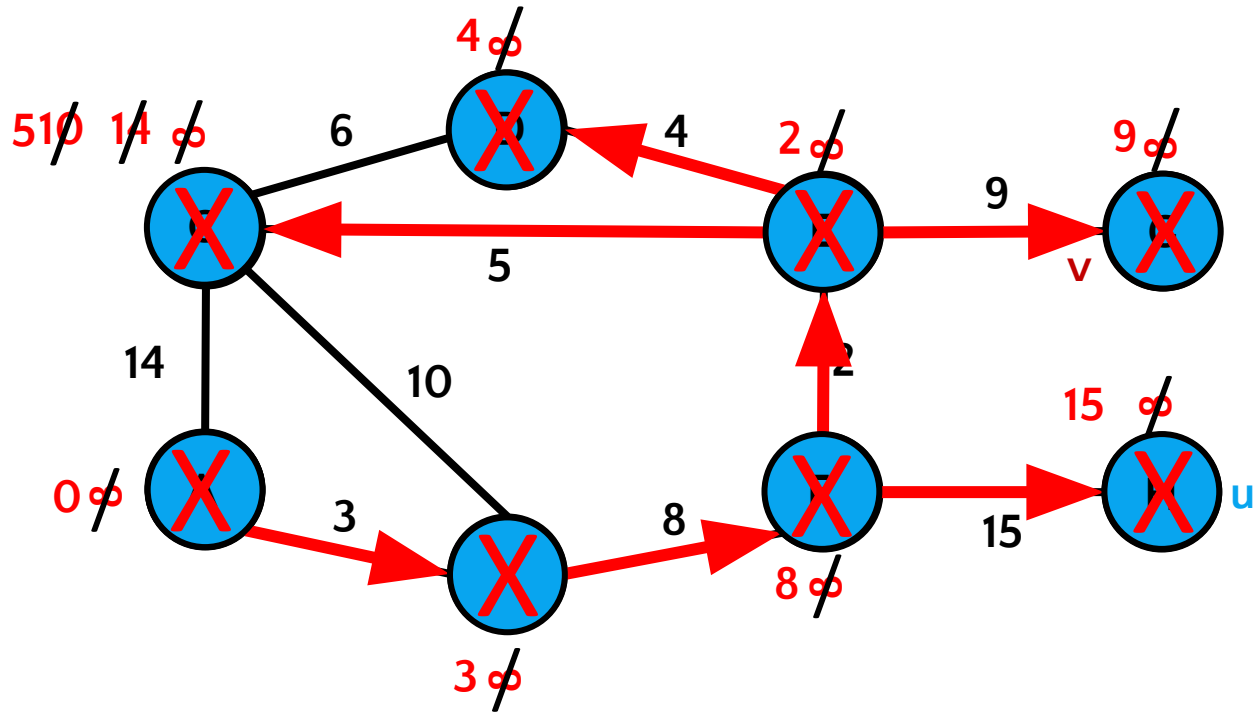
- Set the key of all vertices to INF
- Pick a random vertex as root and do  $\text{key}[\text{root}] := 0$
- Iteration until all vertices are visited
  - $u :=$  unexplored vertex with lowest key
  - $v :=$  unexplored adjacent vertices of  $u$
  - If  $\text{key}(v) > w(u, v)$ 
    - $\text{parent}(v) := u$
    - $\text{key}(v) := w(u, v)$
  - Make  $u$  explored

# PRIM's ALGORITHM (SIMULATION)



- Set the key of all vertices to INF
- Pick a random vertex as root and do  $\text{key}[\text{root}] := 0$
- Iteration until all vertices are visited
  - $u :=$  unexplored vertex with lowest key
  - $v :=$  unexplored adjacent vertices of  $u$
  - If  $\text{key}(v) > w(u, v)$ 
    - $\text{parent}(v) := u$
    - $\text{key}(v) := w(u, v)$
  - Make  $u$  explored

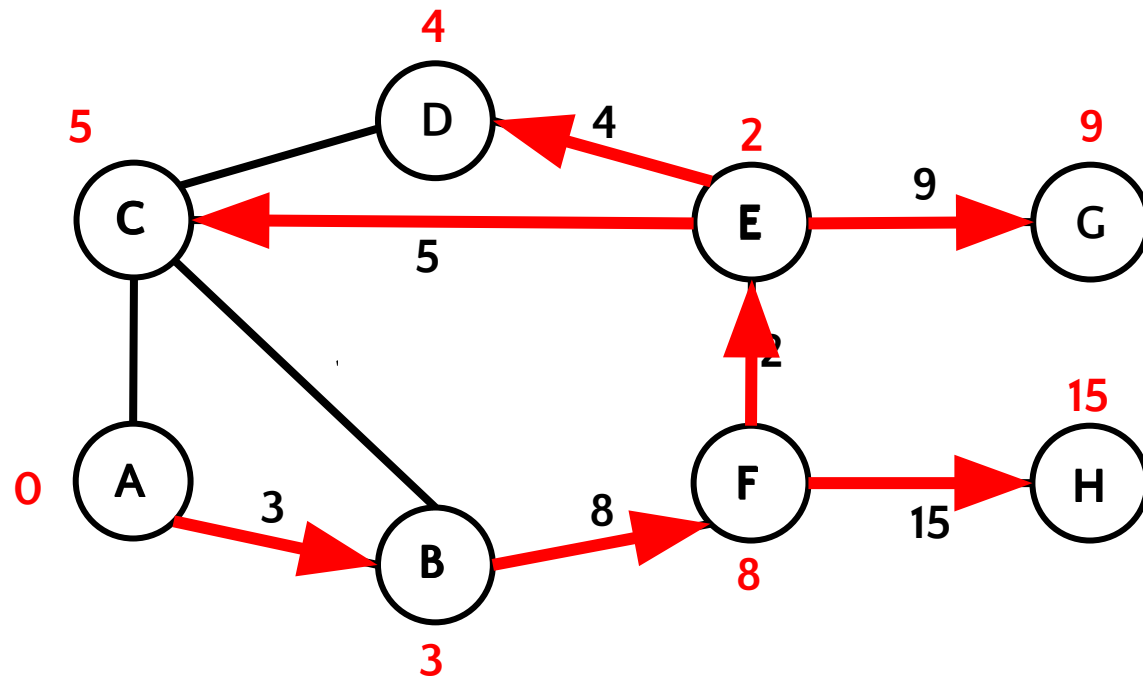
# PRIM's ALGORITHM (SIMULATION)



- Set the key of all vertices to INF
- Pick a random vertex as root and do  $\text{key}[\text{root}] := 0$
- Iteration until all vertices are visited
  - $u :=$  unexplored vertex with lowest key
  - $v :=$  unexplored adjacent vertices of  $u$
  - If  $\text{key}(v) > w(u, v)$ 
    - $\text{parent}(v) := u$
    - $\text{key}(v) := w(u, v)$
  - Make  $u$  explored



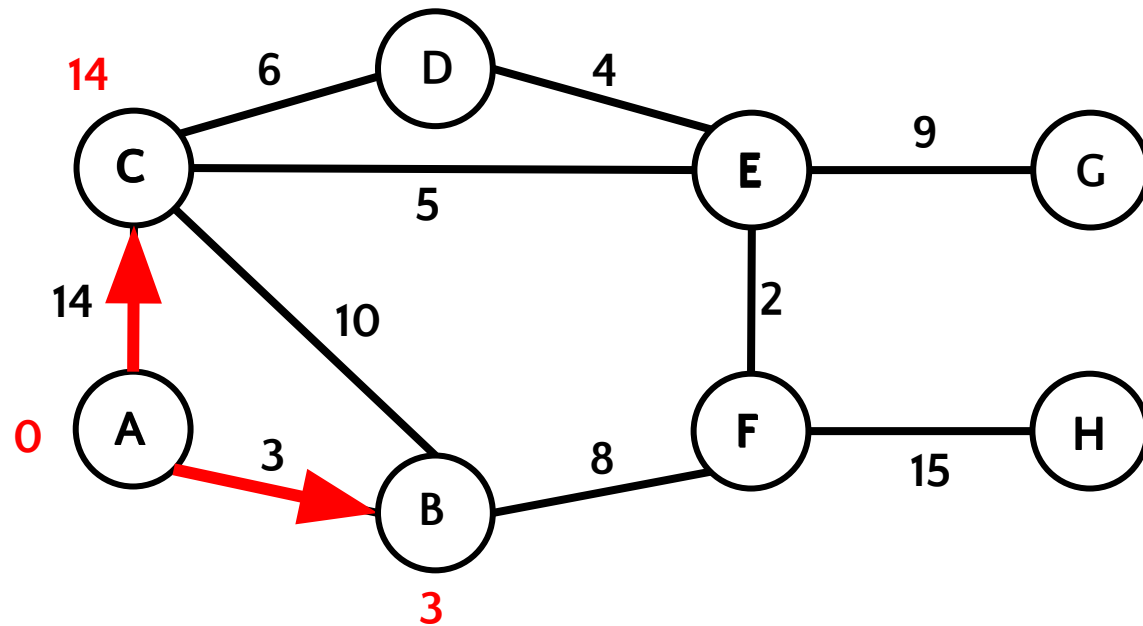
# PRIM's ALGORITHM (SIMULATION)





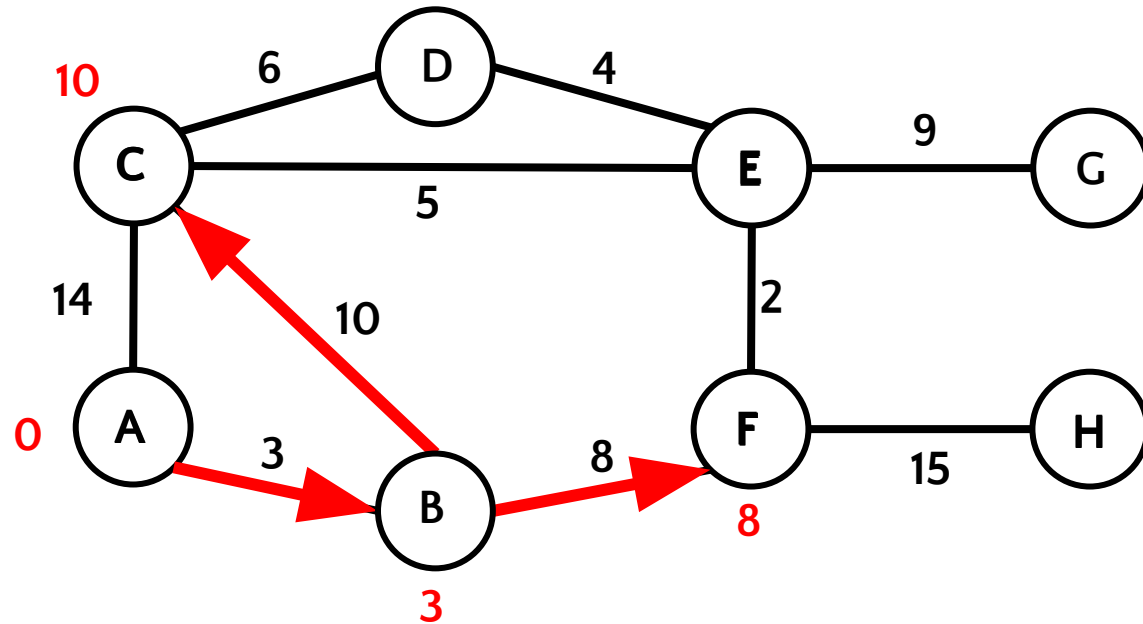


# PRIM's ALGORITHM (GROWTH OF MST)



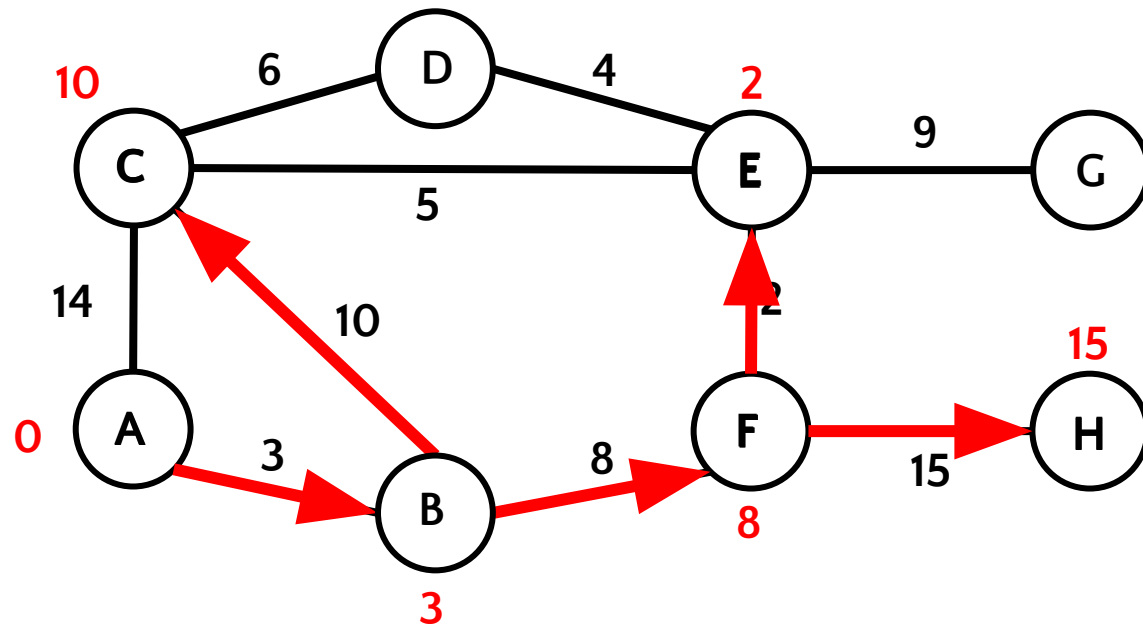


# PRIM's ALGORITHM (GROWTH OF MST)



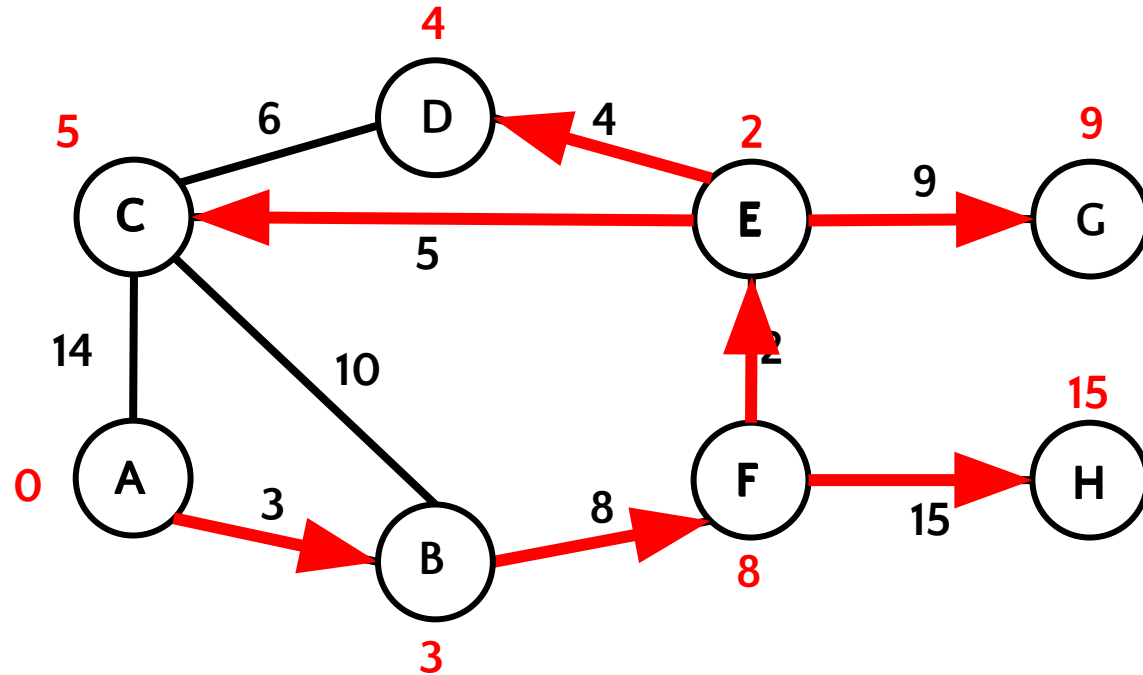


# PRIM's ALGORITHM (GROWTH OF MST)

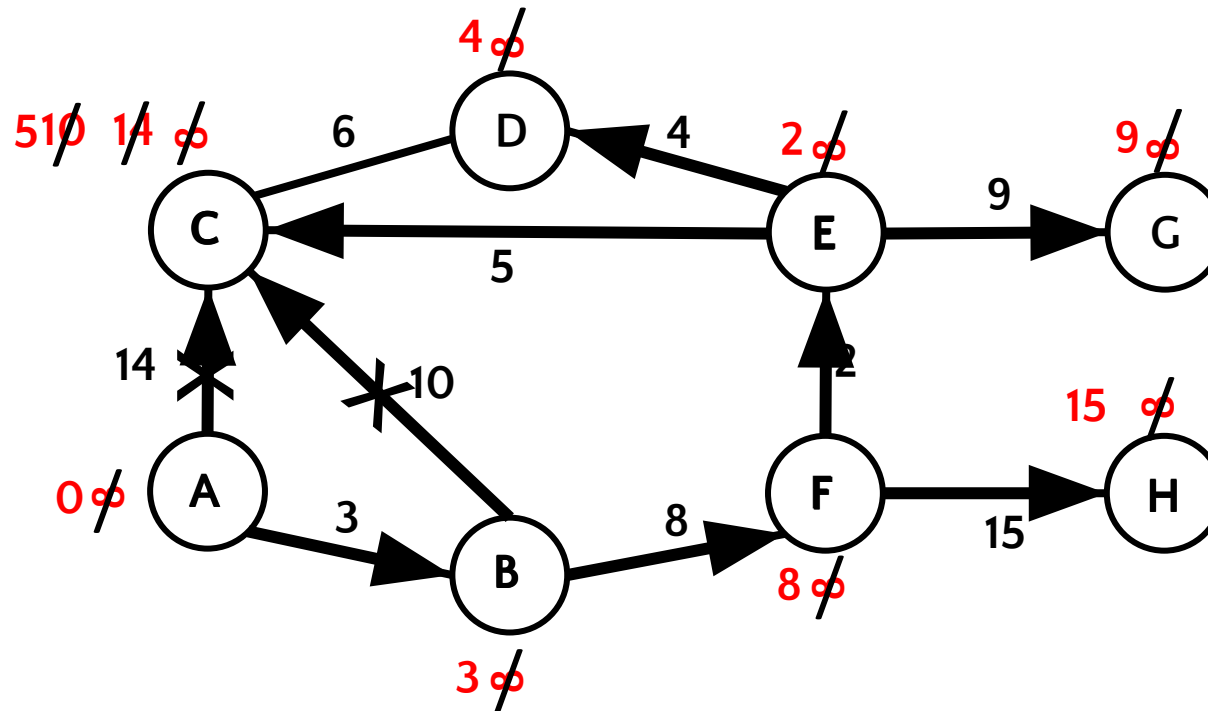




# PRIM's ALGORITHM (GROWTH OF MST)



# HOW TO ANSWER?



Exploration sequence: A → B → F → E → D → C → G → H

Total cost: 46

# PRIM'S ALGORITHM

MST-Prim( $G, w, r$ )

for each  $u \in V$

$key[u] := \infty$

$Q = V[G]$

$key[r] := 0$

→  $decreaseKey(r, 0)$

$parent[r] := NULL$

while( $Q$  not empty)

$u = extractMin(Q)$

    for each  $v \in V$

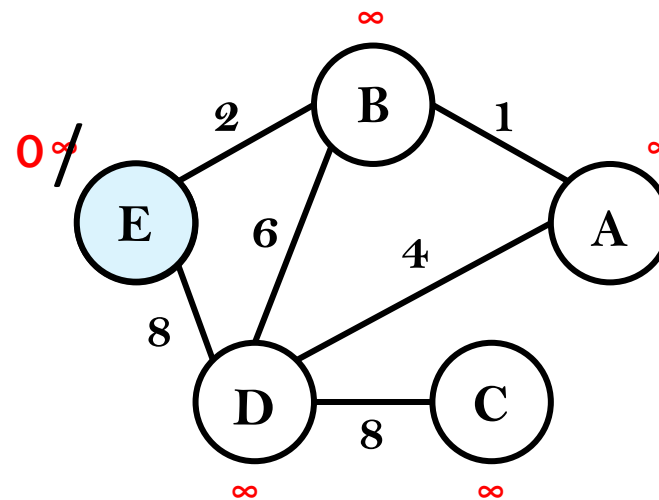
        if ( $v \in Q$  and  $w(u, v) <$

$key[v]$ )

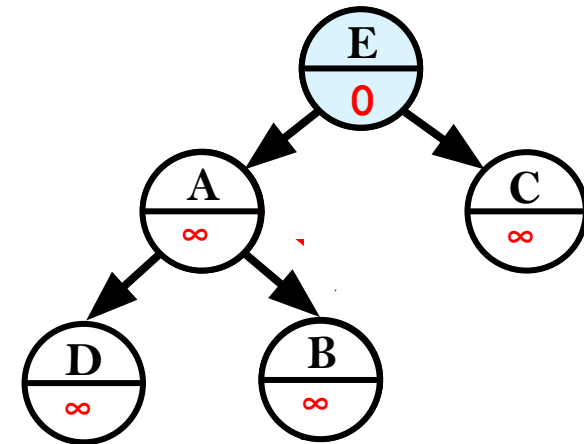
$key[v] := w(u, v)$

$decreaseKey(v, w(u, v))$

Graph ( $G$ )



Priority Queue ( $Q$ )



# PRIM'S ALGORITHM

MST-Prim( $G, w, r$ )

for each  $u \in$

$V[G]$   $key[u] := \infty$

$Q = V[G]$

$key[r] := 0$

decreaseKey( $r, 0$ )

parent[r] := NULL

while( $Q$  not empty)

$u = \text{extractMin}(Q)$

for each  $v \in$

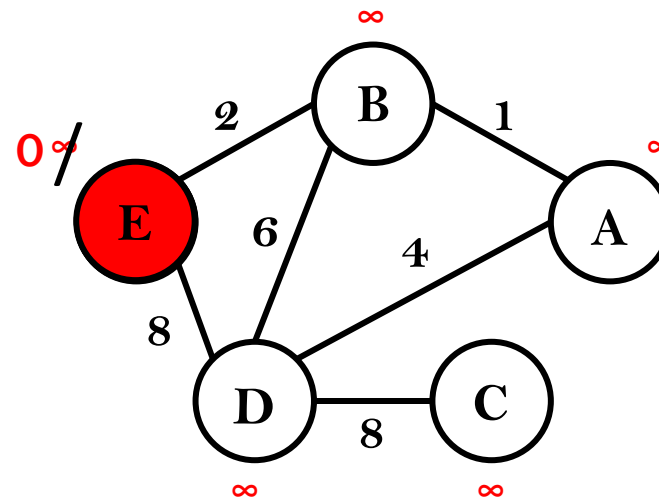
$\text{adj}[u] (v \in Q \text{ and } w(u, v) <$

$key[v])$  parent[v] :=  $u$

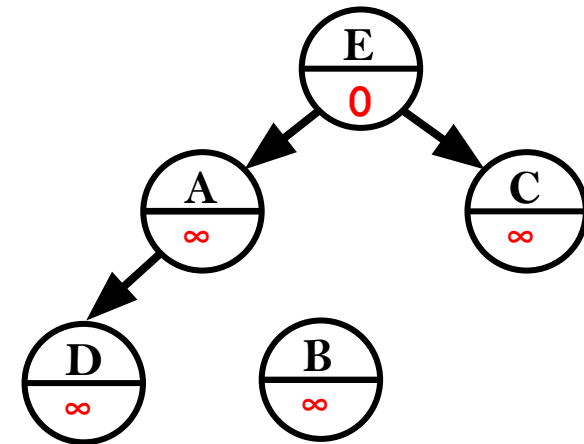
$key[v] := w(u, v)$

decreaseKey( $v, w(u, v)$ )

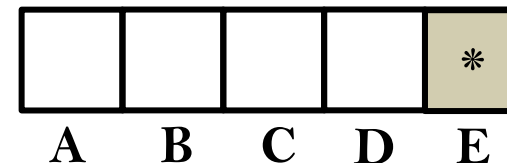
Graph ( $G$ )



Priority Queue ( $Q$ )



parent



$u =$

# PRIM'S ALGORITHM

MST-Prim( $G, w, r$ )

for each  $u \in$

$V[G]$   $key[u] := \infty$

$Q = V[G]$

$key[r] := 0$

decreaseKey( $r, 0$ )

parent[r] := NULL

while( $Q$  not empty)

$u = \text{extractMin}(Q)$

for each  $v \in$

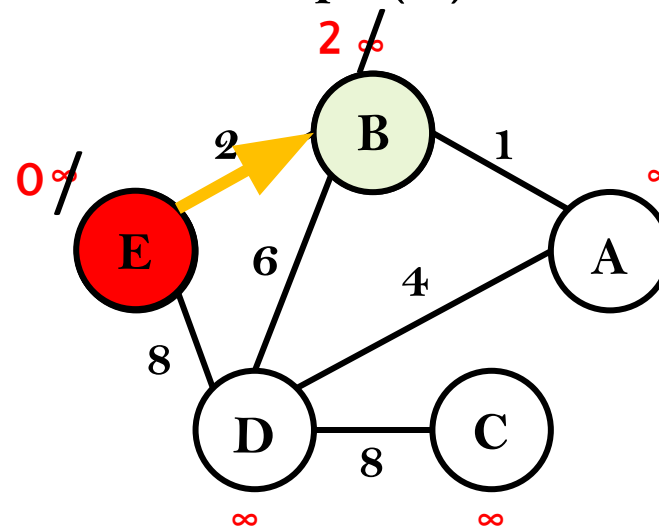
$\{v \in Q \text{ and } w(u, v) <$

$key[v]\}$  parent[v] :=  $u$

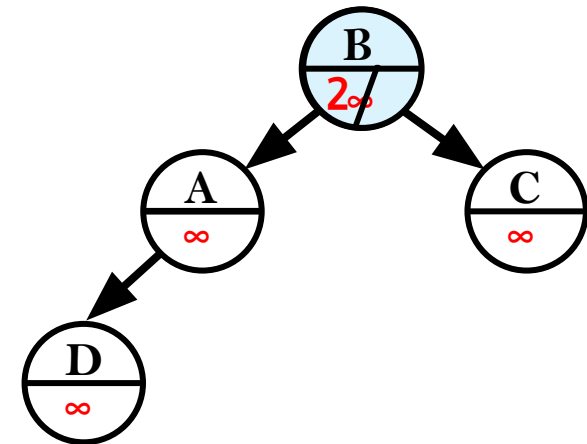
key[v] :=  $w(u, v)$

→ decreaseKey( $v, w(u, v)$ )

Graph ( $G$ )



Priority Queue ( $Q$ )



parent

|   |   |   |   |   |
|---|---|---|---|---|
|   | E |   |   | * |
| A | B | C | D | E |

$u =$



# PRIM'S ALGORITHM

MST-Prim( $G, w, r$ )

for each  $u \in$

$V[G]$   $key[u] := \infty$

$Q = V[G]$

$key[r] := 0$

decreaseKey( $r, 0$ )

parent[r] := NULL

while( $Q$  not empty)

$u = \text{extractMin}(Q)$

for each  $v \in$

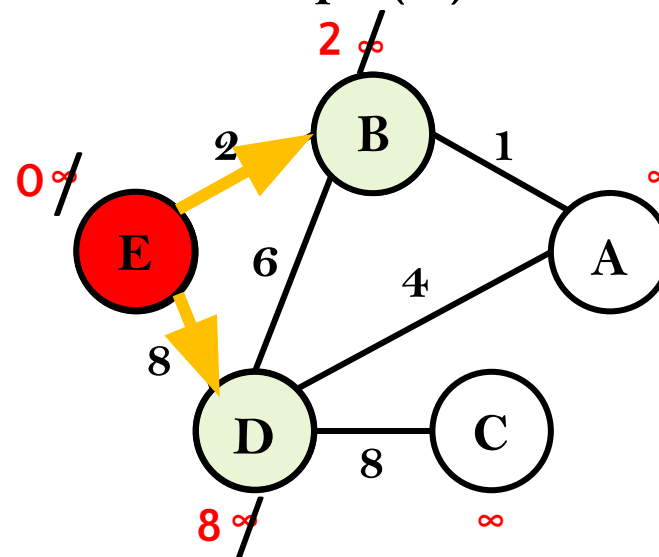
$\{v \mid (v \in Q \text{ and } w(u, v) <$

$key[v])$  parent[v] :=  $u$

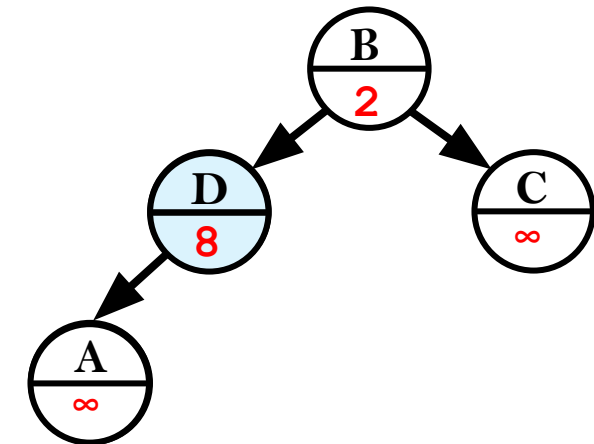
key[v] :=  $w(u, v)$

→ decreaseKey( $v, w(u, v)$ )

Graph ( $G$ )



Priority Queue ( $Q$ )



$u =$

parent

|   |   |   |   |   |
|---|---|---|---|---|
|   | E |   | E | * |
| A | B | C | D | E |

# PRIM'S ALGORITHM

MST-Prim( $G, w, r$ )

for each  $u \in$

$V[G]$   $key[u] := \infty$

$Q = V[G]$

$key[r] := 0$

decreaseKey( $r, 0$ )

parent[r] := NULL

while( $Q$  not empty)

➔  $u = \text{extractMin}(Q)$

for each  $v \in$

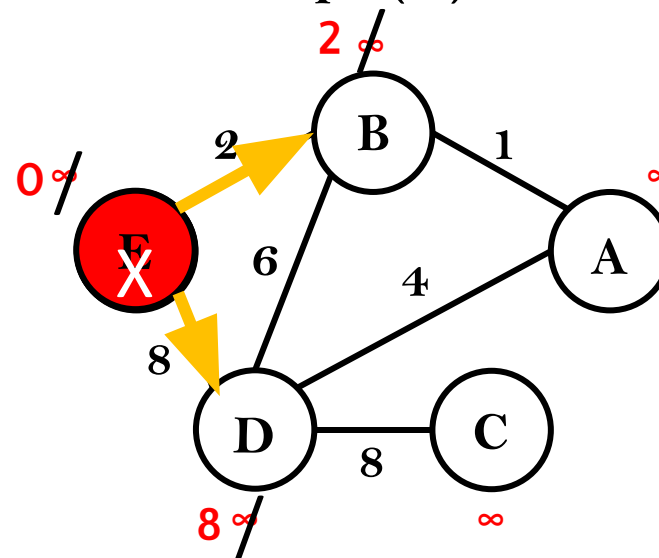
$\text{adj}[u] (v \in Q \text{ and } w(u, v) <$

$key[v])$  parent[v] :=  $u$

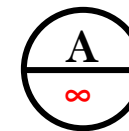
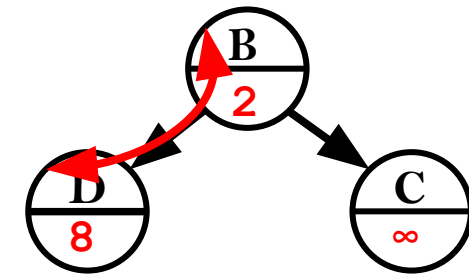
key[v] :=  $w(u, v)$

decreaseKey( $v, w(u, v)$ )

Graph ( $G$ )



Priority Queue ( $Q$ )



parent

|   |   |   |   |   |
|---|---|---|---|---|
|   | E |   | E | * |
| A | B | C | D | E |

$u =$

# PRIM'S ALGORITHM

MST-Prim( $G, w, r$ )

for each  $u \in$

$V[G]$   $key[u] := \infty$

$Q = V[G]$

$key[r] := 0$

$decreaseKey(r, 0)$

$parent[r] := NULL$

while( $Q$  not empty)

$u = extractMin(Q)$

for each  $v \in$

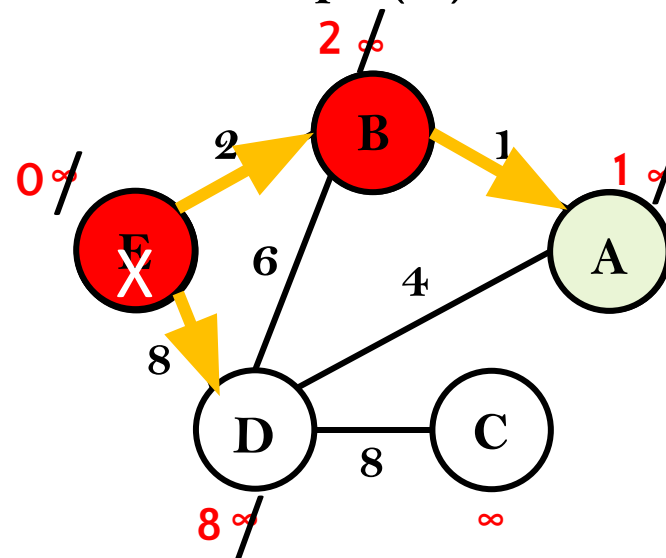
$\{v \in Q \text{ and } w(u, v) <$

$key[v]\}$   $parent[v] := u$

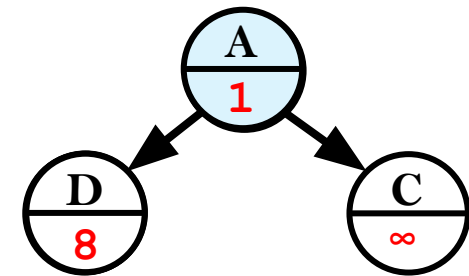
$key[v] := w(u, v)$

$\rightarrow decreaseKey(v, w(u, v))$

Graph ( $G$ )



Priority Queue ( $Q$ )



parent

|   |   |   |   |   |
|---|---|---|---|---|
| B | E |   | E | * |
| A | B | C | D | E |

$u =$

# PRIM'S ALGORITHM

MST-Prim( $G, w, r$ )

for each  $u \in$

$V[G]$   $key[u] := \infty$

$Q = V[G]$

$key[r] := 0$

$decreaseKey(r, 0)$

$parent[r] := NULL$

while( $Q$  not empty)

$u = extractMin(Q)$

for each  $v \in$

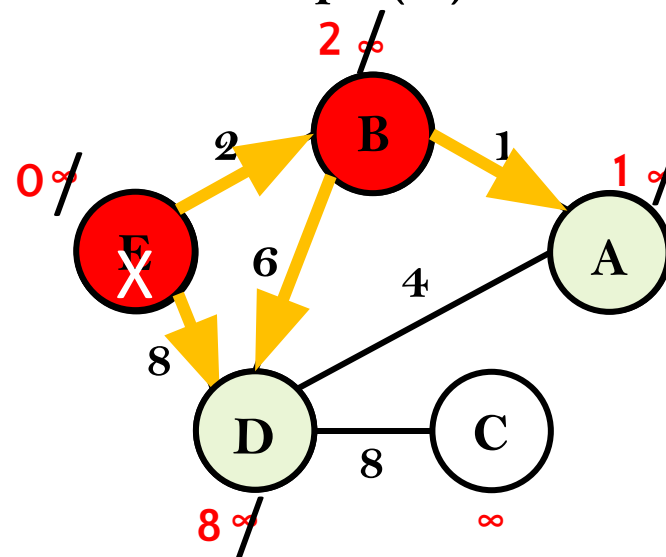
$\{v \in Q \text{ and } w(u, v) <$

$key[v]\}$   $parent[v] := u$

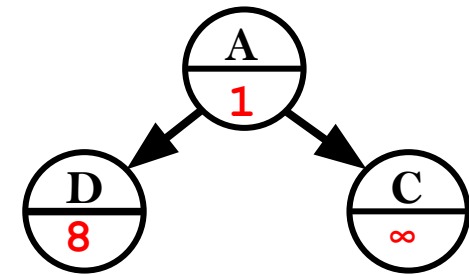
$key[v] := w(u, v)$

$decreaseKey(v, w(u, v))$

Graph ( $G$ )



Priority Queue ( $Q$ )



parent

|   |   |   |   |   |
|---|---|---|---|---|
| B | E |   | E | * |
| A | B | C | D | E |

$u =$

# PRIM'S ALGORITHM

MST-Prim( $G, w, r$ )

for each  $u \in$

$V[G]$   $key[u] := \infty$

$Q = V[G]$

$key[r] := 0$

decreaseKey( $r, 0$ )

parent[r] := NULL

while( $Q$  not empty)

$u = \text{extractMin}(Q)$

    for each  $v \in$

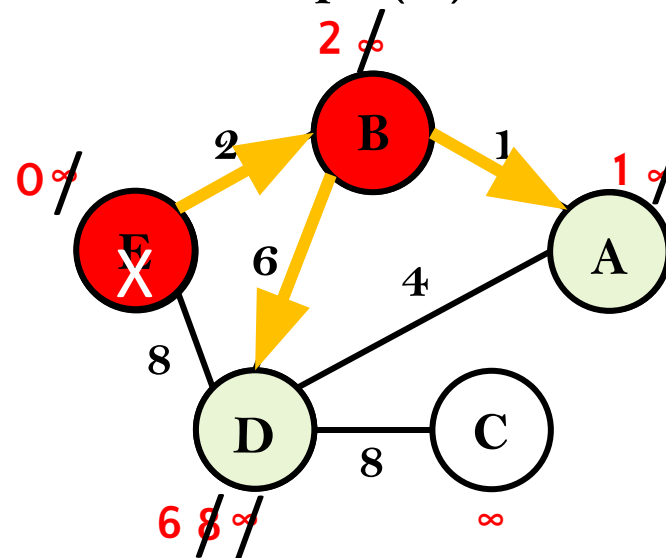
$\{v \in Q \text{ and } w(u, v) <$

$key[v]\}$   $parent[v] := u$

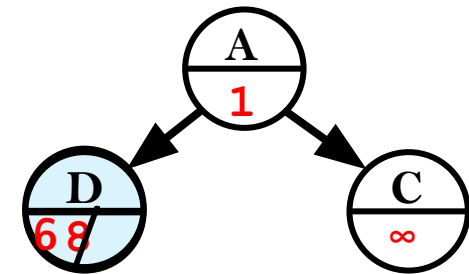
$key[v] := w(u, v)$

        ➔  $\text{decreaseKey}(v, w(u, v))$

Graph ( $G$ )



Priority Queue ( $Q$ )



parent

|   |   |   |   |   |
|---|---|---|---|---|
| B | E |   | B | * |
| A | B | C | D | E |

$u =$

# PRIM'S ALGORITHM

MST-Prim(G, w, r)

for each  $u \in$

$V[G]$   $key[u] := \infty$

$Q = V[G]$

$key[r] := 0$

$decreaseKey(r, 0)$

$parent[r] := NULL$

while( $Q$  not empty)

$u = extractMin(Q)$

for each  $v \in$

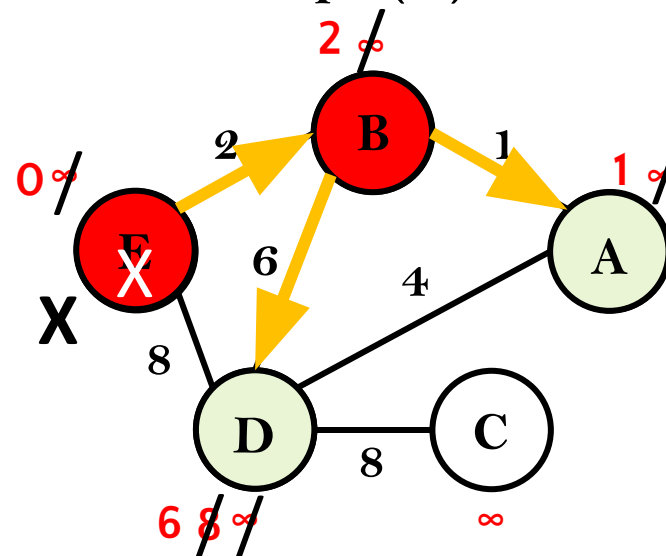
$V[G]$  if ( $v \in Q$  and  $w(u, v) <$

$key[v]$ )  $parent[v] := u$

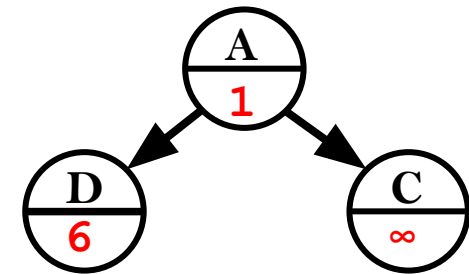
$key[v] := w(u, v)$

$decreaseKey(v, w(u, v))$

Graph (G)



Priority Queue (Q)



$u =$

parent

|   |   |   |   |   |
|---|---|---|---|---|
| B | E |   | B | * |
| A | B | C | D | E |

# PRIM'S ALGORITHM

MST-Prim(G, w, r)

for each  $u \in$

$V[G]$   $key[u] := \infty$

$Q = V[G]$

$key[r] := 0$

$decreaseKey(r, 0)$

$parent[r] := NULL$

while( $Q$  not empty)

➔  $u = extractMin(Q)$

for each  $v \in$

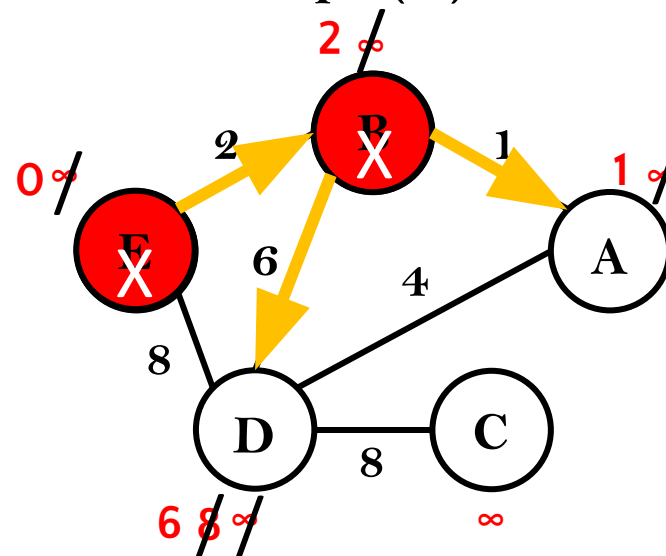
$adj[u]$  ( $v \in Q$  and  $w(u, v) <$

$key[v]$ )  $parent[v] := u$

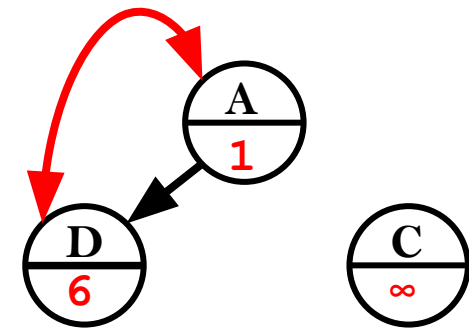
$key[v] := w(u, v)$

$decreaseKey(v, w(u, v))$

Graph (G)



Priority Queue (Q)



parent

|   |   |   |   |   |
|---|---|---|---|---|
| B | E |   | B | * |
| A | B | C | D | E |

$u =$

# PRIM'S ALGORITHM

MST-Prim( $G, w, r$ )

for each  $u \in$

$V[G]$   $key[u] := \infty$

$Q = V[G]$

$key[r] := 0$

decreaseKey( $r, 0$ )

parent[r] := NULL

while( $Q$  not empty)

$u = \text{extractMin}(Q)$

for each  $v \in$

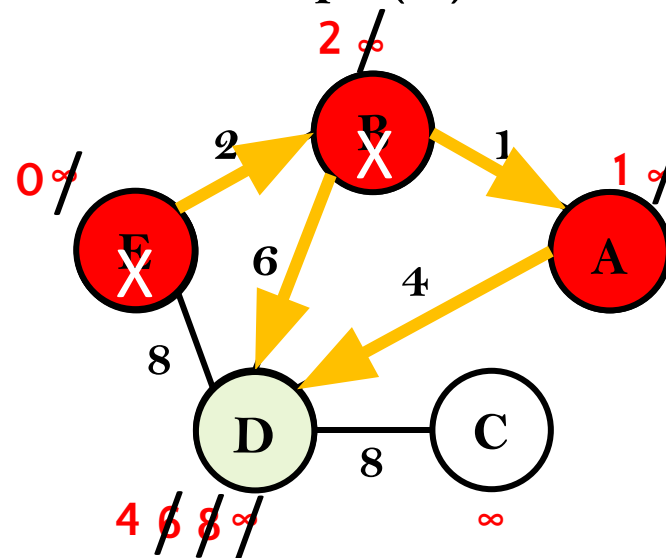
$\{v \in Q \text{ and } w(u, v) <$

$key[v]\}$  parent[v] :=  $u$

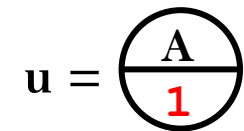
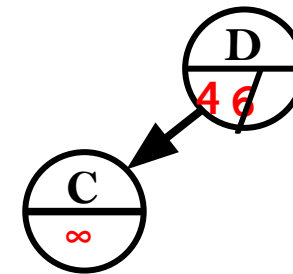
key[v] :=  $w(u, v)$

➔ decreaseKey( $v, w(u, v)$ )

Graph ( $G$ )



Priority Queue ( $Q$ )



parent

|   |   |   |   |   |
|---|---|---|---|---|
| B | E |   | A | * |
| A | B | C | D | E |



# PRIM'S ALGORITHM

MST-Prim(G, w, r)

for each  $u \in$

$V[G]$   $key[u] := \infty$

$Q = V[G]$

$key[r] := 0$

decreaseKey( $r, 0$ )

parent[r] := NULL

while( $Q$  not empty)

$u = \text{extractMin}(Q)$

for each  $v \in$

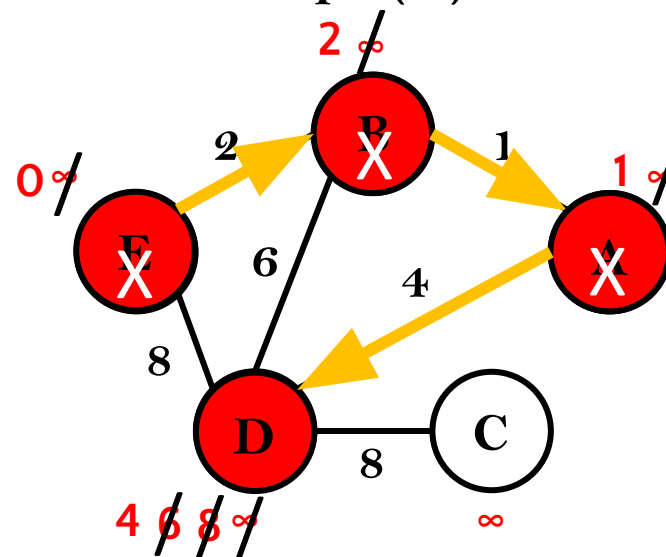
$V[G]$  ( $v \in Q$  and  $w(u, v) <$

$key[v]$ )  
parent[v] :=  $u$

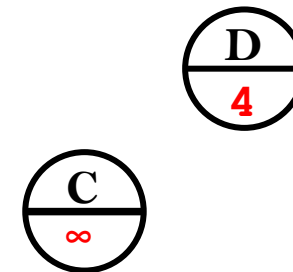
$key[v] := w(u, v)$

decreaseKey( $v, w(u, v)$ )

Graph (G)



Priority Queue (Q)



parent

|   |   |   |   |   |
|---|---|---|---|---|
| B | E |   | A | * |
| A | B | C | D | E |

$u =$

# PRIM'S ALGORITHM

MST-Prim( $G, w, r$ )

for each  $u \in$

$V[G]$   $key[u] := \infty$

$Q = V[G]$

$key[r] := 0$

decreaseKey( $r, 0$ )

parent[r] := NULL

while( $Q$  not empty)

$u = \text{extractMin}(Q)$

for each  $v \in$

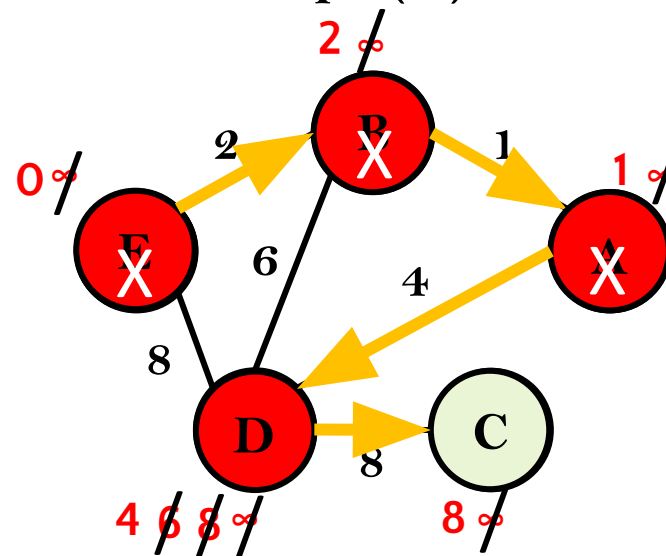
$\text{adj}[u] (v \in Q \text{ and } w(u, v) <$

$key[v])$  parent[v] :=  $u$

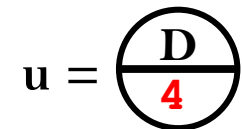
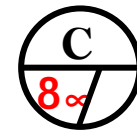
key[v] :=  $w(u, v)$

➔ decreaseKey( $v, w(u, v)$ )

Graph ( $G$ )



Priority Queue ( $Q$ )



parent

|   |   |   |   |   |
|---|---|---|---|---|
| B | E | D | A | * |
| A | B | C | D | E |

# PRIM'S ALGORITHM

MST-Prim(G, w, r)

for each  $u \in$

$V[G]$   $key[u] := \infty$

$Q = V[G]$

$key[r] := 0$

$decreaseKey(r, 0)$

$parent[r] := NULL$

while( $Q$  not empty)

$u = extractMin(Q)$

for each  $v \in$

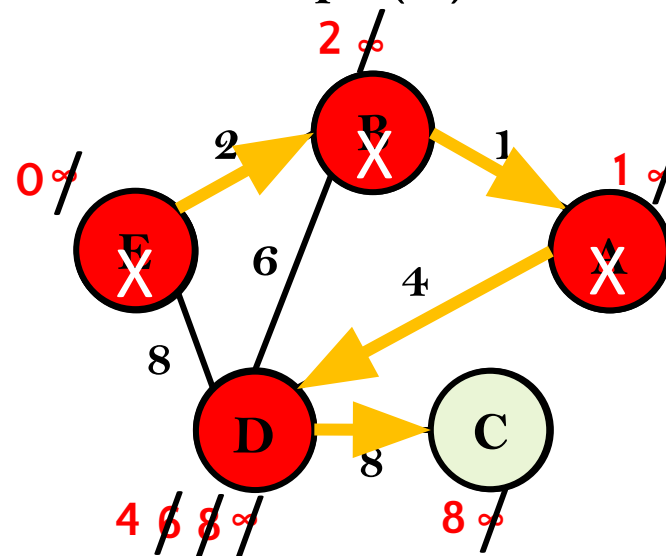
$adj[u]$  ( $v \in Q$  and  $w(u, v) <$

$key[v]$ )  $parent[v] := u$

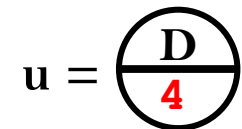
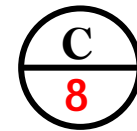
$key[v] := w(u, v)$

➔  $decreaseKey(v, w(u, v))$

Graph (G)



Priority Queue (Q)



parent

|   |   |   |   |   |
|---|---|---|---|---|
| B | E | D | A | * |
| A | B | C | D | E |

# PRIM'S ALGORITHM

MST-Prim(G, w, r)

for each  $u \in$

$V[G]$   $key[u] := \infty$

$Q = V[G]$

$key[r] := 0$

decreaseKey( $r, 0$ )

parent[r] := NULL

→ while(Q not empty)  
DONE!  $u = \text{extractMin}(Q)$

for each  $v \in$

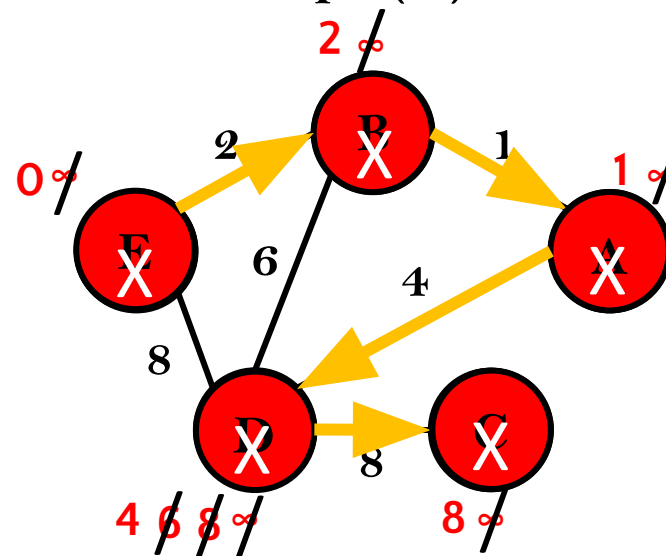
$\text{adj}[u] (v \in Q \text{ and } w(u, v) <$

$key[v])$   
parent[v] := u

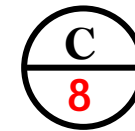
key[v] :=  $w(u, v)$

decreaseKey( $v, w(u, v)$ )

Graph (G)



Priority Queue (Q)



parent

|   |   |   |   |   |
|---|---|---|---|---|
| B | E | D | A | * |
| A | B | C | D | E |

u =



# PRIM's ALGORITHM

---

MST-Prim( $G, w, r$ )

for each  $u \in$

$V[G]$   $key[u] := \infty$

$Q = V[G]$

$key[r] := 0$

decreaseKey( $r, 0$ )

parent[r] := NULL

while( $Q$  not empty)

$u = \text{extractMin}(Q)$

    for each  $v \in$

        adj[u] ( $v \in Q$  and  $w(u, v) <$

$key[v]$ )

        parent[v] :=  $u$

$key[v] := w(u, v)$

        decreaseKey( $v, w(u, v)$ )



# Thank You!

---